

Week 5 — Building a Minimal CPU

Curriculum alignment: Stage 2 — CPU Architecture and Instruction Execution · The Minimal CPU

Target time: approximately 3 hours

- Historical problem and stored programs: about 25 minutes
- Clock, state, datapath, and control: about 55 minutes
- Tiny ISA design and encoding: about 35 minutes
- Cycle-by-cycle trace: about 30 minutes
- Python build, observation, and mastery work: about 35 minutes

We can already build logic that adds two bit patterns. Why does that not already give us a computer that can calculate $3 + 5$?

An adder answers one fixed question whenever its inputs settle. It does not remember the previous answer, choose its own next operands, or decide that the next operation should be a store rather than another addition. A programmable machine needs a controlled sequence of remembered states.

This lesson builds that machine without hiding the transitions behind the phrase “the CPU executes an instruction.” We will design an explicit four-instruction architecture, encode a program as bytes, and account for what changes—and what does not—at each clock edge.

The central distinction is:

Combinational circuitry computes between state updates. A clock edge coordinates when selected state elements capture new values. The clock does not create the computation.

1. The problem that fixed-purpose calculators could not solve

A gate network can implement a truth table. An adder network can continuously produce $A + B$. A mechanical calculator or a relay machine can similarly be wired so its structure enforces an operation.

But suppose the desired procedure is:

```
put 3 somewhere
put 5 somewhere else
add the two stored values
save the result
stop
```

Four new problems appear:

1. **Storage:** where do operands and intermediate results persist?
2. **Sequence:** which operation happens first, and which follows?
3. **Selection:** how does the same hardware perform different operations?
4. **Repetition:** how can changing a stored description change the procedure without rewiring the machine?

Early machines often received control from plugboards, switches, or punched media. Those approaches can be programmable, but instructions are supplied by a mechanism distinct from the machine's working memory. The stored-program idea makes a more powerful move: encode instructions as bit patterns and keep them in addressable electronic memory.

The Manchester Small-Scale Experimental Machine, or “Baby,” demonstrated this idea in working electronic hardware. Its first successful program was stored in memory along with data. It was a deliberately small experimental machine, largely built to test a practical electronic store, yet it established the essential loop:

```
address an instruction
  → fetch its stored bits
  → interpret those bits as control
  → transform machine state
  → select the next instruction
```

This is why a stored program is more than “data in memory.” The machine has a path that treats selected memory bits as instructions and lets those instructions determine future state transitions.

2. State is the CPU's memory of the past

A **state element** preserves a value after the signals that produced it have changed. Registers, flip-flops, and memory cells are stateful. An adder, decoder, or multiplexer considered by itself is combinational: its settled output is a function of its current inputs.

For our minimal CPU, the important state is:

PC	address of the next instruction to fetch
IR	instruction byte currently being executed
R0..R3	four general-purpose registers
data memory	sixteen stored data bytes
phase	whether the next cycle is FETCH or EXECUTE
halted	whether state updates should stop

There are two useful categories here.

Architectural state

Architectural state is the state software can observe according to the instruction-set contract. In our tiny design it includes R0..R3, data memory, and the sequencing behavior represented by the PC. If two implementations finish each instruction with the same architectural state, software sees the same machine.

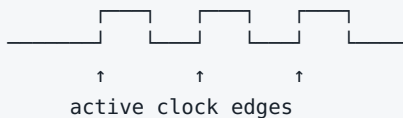
Microarchitectural state

Microarchitectural state exists because of how this particular CPU is built. Our IR and phase register are examples. Another implementation of the same ISA might use more pipeline registers, no separately visible IR, or several internal steps per instruction. Those differences need not change the ISA.

The boundary can be subtle. A debugger may expose an implementation's PC or internal registers, but that does not automatically make every internal latch part of the ISA contract. Always ask: **must every conforming implementation present this behavior to software, or is this one hardware design's method?**

3. What the clock does—and does not do

A clock is a periodic timing signal, often drawn as alternating LOW and HIGH:



An edge-triggered register samples its input at an active edge and preserves that sampled value until a later permitted edge.

Between edges, signals propagate through combinational paths:

```
old register outputs
    ↓
decoder, multiplexers, ALU, address logic
    ↓
candidate next values settle
    ↓
next clock edge captures selected candidates
```

The clock does **not** add numbers. It does not decode opcodes. It does not push data through wires. Transistor networks perform that combinational work in response to their inputs. The clock provides coordinated moments at which state elements commit selected results.

This leads to a precise account of adjacent cycles:

1. Just after edge **N**, registers expose their newly stored outputs.
2. During the interval after edge **N**, combinational circuitry reacts and settles toward candidate next values.
3. Setup-time requirements demand that required candidates be stable before edge **N+1**.
4. At edge **N+1**, enabled state elements capture those candidates.
5. Just after that edge, their changed outputs begin another round of propagation.

The clock period must be long enough for the slowest required path, plus the timing margins of the state elements. A faster clock is not free: if the next edge arrives before the combinational result is valid, the CPU may capture the wrong state.

A snapshot is not the whole cycle

When a trace prints one line per edge, it hides continuous activity between the lines. “Cycle 4: R0 becomes 8” really means:

- before the edge, R0=3 and R1=5 drive ALU inputs;
- the ADD network settles to 8;
- control asserts the write enable for R0;
- at the edge, R0 captures 8;
- after the edge, the register's output changes to 8.

The clock edge commits the answer; it did not produce it.

4. The core components

Program counter (PC)

The PC is a register holding the address of the **next instruction to fetch**. During our FETCH cycle, instruction memory is read at PC, and the candidate next PC is $PC + 1$. At the fetch edge, the IR captures the instruction and the PC captures that incremented address.

This convention matters. During the following EXECUTE cycle, the PC already points past the instruction in the IR.

Instruction memory

Instruction memory maps an address to an instruction byte. We model it as read-only while the CPU runs:

```
instruction_address = PC
instruction_byte     = instruction_memory[PC]
```

Real machines may share physical storage paths for code and data or use separate caches and buses. Separate instruction and data arrays make our first datapath easier to see; that is a microarchitectural choice.

Instruction register (IR)

The IR preserves the fetched byte while the PC moves on and the decoder uses the byte's fields. Without an IR in this two-cycle design, changing the PC during fetch would also change the instruction-memory output being decoded.

Register file

The register file holds four eight-bit values: R0, R1, R2, and R3. Read ports expose selected values to combinational logic. A write port accepts a destination number, value, and write-enable. On an active edge with writing enabled, exactly the selected register captures the result.

Arithmetic and logic unit (ALU)

Our minimal ALU only needs addition, plus pass-through behavior useful for displaying immediate and store values. It is combinational:

```
ALU_out = operation(ALU_A, ALU_B)
```

The ALU output is not automatically remembered. A register write or memory write must be enabled at an edge for the result to become state.

Data memory

Data memory contains sixteen eight-bit locations. **STORE** selects an address, places a register value on the write-data path, and asserts memory write enable. Our simulator commits that write during the EXECUTE state update.

Control unit

The control unit examines the current phase and, during EXECUTE, the IR's opcode and fields. It emits decisions such as:

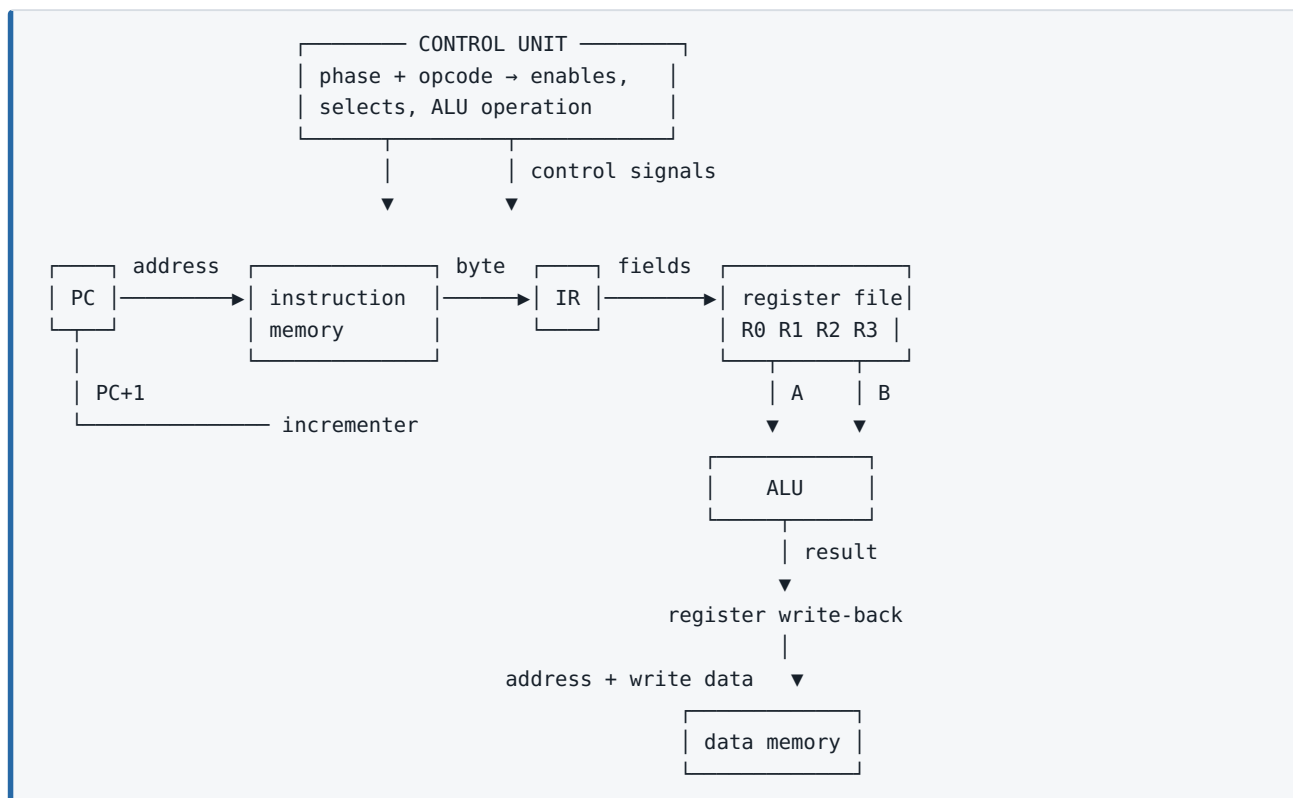
ir_load	should IR capture instruction memory?
pc_increment	should PC capture PC + 1?
alu_op	which combinational ALU function?
reg_write	should a register capture ALU_out?
mem_write	should data memory capture a value?
halt	should the halted state be set?

Control does not carry the operand value itself. It opens and closes paths, selects operations, and enables state updates.

5. Datapath versus control

The **datapath** contains the places where values are stored and transformed: PC, IR, register file, buses, multiplexers, ALU, and memory interfaces.

The **control path** decides which datapath action is allowed now.



The diagram is a logical block diagram, not a claim that values flow in a single direction on every real wire. A practical register file has multiple ports; multiplexers select among sources; wires fan out; timing and electrical loading matter.

Why make the control/datapath split? Because the same expensive value-moving hardware can be reused. The ADD instruction selects two registers and enables write-back. STORE selects a register and enables a memory write. Different control patterns cause one datapath to realize different state transitions.

6. An explicit tiny four-instruction ISA

We will call the architecture **Tiny-8**.

Programmer-visible contract

- Instructions are one byte each.
- PC addresses instruction bytes and normally advances by one.
- Four registers, R0..R3, each hold an unsigned eight-bit pattern.
- Addition wraps modulo 256.
- Data memory has sixteen byte locations, addresses 0x0..0xF.
- Execution begins with PC=0, all registers and data memory zero.
- The four instructions are LDI, ADD, STORE, and HALT.

Encoding

Every instruction begins with a two-bit opcode:

bits 7..6	opcode
00	LDI
01	ADD
10	STORE
11	HALT

The complete formats are:

LDI Rd, imm4	00 dd iiii
ADD Rd, Rs	01 dd ss 00
STORE Rs, addr4	10 ss aaaa
HALT	11 000000

dd or ss selects one of four registers:

00 → R0 01 → R1 10 → R2 11 → R3

Instruction behavior is:

LDI Rd, imm4	$R[dd] \leftarrow \text{zero_extend}(\text{imm4})$
ADD Rd, Rs	$R[dd] \leftarrow (R[dd] + R[ss]) \bmod 256$
STORE Rs, addr4	$\text{MEM}[\text{addr4}] \leftarrow R[ss]$
HALT	$\text{halted} \leftarrow 1$

Some encodings contain reserved zeros. For ADD, bits 1..0 must be 00. For HALT, bits 5..0 must be zero. Reserving patterns makes malformed instructions detectable and leaves design space for later extensions.

Encode the 3 + 5 program

address	assembly	fields	binary	hex
0	LDI R0, 3	00 00 0011	00000011	0x03
1	LDI R1, 5	00 01 0101	00010101	0x15
2	ADD R0, R1	01 00 01 00	01000100	0x44
3	STORE R0, 0	10 00 0000	10000000	0x80
4	HALT	11 000000	11000000	0xC0

The program is therefore the byte sequence:

03 15 44 80 C0

The byte 0x44 does not intrinsically mean ADD R0, R1. Under ASCII it is D; under unsigned interpretation it is 68. Tiny-8's ISA encoding and a Tiny-8 decoder give it instruction meaning.

7. Four layers that must not be collapsed

1. ISA specification

The ISA is a contract written for programmers and hardware implementers. It says which architectural state exists and how each legal instruction changes that state. ADD R0, R1 must leave R0 holding the wrapped sum.

2. Instruction bytes

0x44 is a stored pattern selected by the PC. It is an instance of the encoding. The storage cell does not carry a tag saying “instruction.”

3. Hardware decoder

The decoder is combinational circuitry that recognizes opcode 01, extracts dd and ss, selects register ports, chooses ADD, and prepares a write enable. It physically responds to signals; it does not understand the programmer's intent.

4. Microarchitecture

The microarchitecture is the organization that fulfills the ISA: this two-cycle FETCH/EXECUTE controller, its IR, buses, register file, and timing. A different Tiny-8 CPU could fetch and execute in one long cycle, use several short cycles, or pipeline instructions. If the ISA-visible results agree at the required boundaries, all can implement the same ISA.

Keep the chain explicit:

```
ISA contract
  ↓ defines legal encodings and effects
instruction byte in memory
  ↓ drives
physical decoder and control signals
  ↓ steer
one chosen microarchitecture
  ↓ commits
architectural state transition
```

8. Fetch-decode-execute as physical and stateful events

“Fetch, decode, execute” is a conceptual sequence, not a universal promise of three clock cycles. In our microarchitecture, decoding is combinational during the EXECUTE interval, so one instruction takes two state-update cycles.

FETCH interval and edge

Before the edge:

```
PC output → instruction-memory address
instruction-memory data → candidate IR input
PC output → incrementer → candidate PC input
control: ir_load=1, pc_increment=1
```

At the edge:

```
IR ← instruction_memory[old PC]
PC ← old PC + 1
phase ← EXECUTE
```

Registers and data memory do not change.

EXECUTE interval and edge

Between edges, the IR's bits continuously drive the decoder. For an ADD:

```
IR opcode → control chooses ADD and reg_write
IR dd     → destination/read port selects R0
IR ss     → source read port selects R1
R0, R1    → ALU inputs
ALU sum   → candidate register-file write data
```

At the edge:

```
R[dd] ← ALU_out
phase ← FETCH
```

PC and IR remain unchanged during this edge. The IR may continue holding the old instruction while the phase change causes fetch controls to become active.

For a STORE, data memory changes instead of a register. For HALT, only the halted state changes. **LDI** routes its immediate field through a zero-extension path and writes the selected register.

What “decode” changes

Combinational decoding usually changes signals, not remembered state. As IR bits settle, control wires settle. No separate decode register exists in this minimal design. If we inserted one, decoding could become its own clocked stage; that would be a microarchitectural change.

9. Trace the program cycle by cycle

Initial state:

```
PC=0  IR=0x00  R0=0 R1=0 R2=0 R3=0  MEM[0]=0  phase=FETCH
```

Instruction 1: **LDI R0, 3**

Cycle 1 — FETCH

Between edges, **PC=0** addresses instruction memory, which presents **0x03**. The incrementer presents **1**.

At the edge:

```
IR: 0x00 → 0x03
PC: 0 → 1
phase: FETCH → EXECUTE
```

Cycle 2 — EXECUTE

The decoder sees opcode **00**, destination **R0**, immediate **3**. The immediate path presents **3**, and control asserts **reg_write**.

At the edge:

```
R0: 0 → 3  
phase: EXECUTE → FETCH
```

PC remains 1; IR remains 0x03.

Instruction 2: LDI R1, 5

Cycle 3 — FETCH

Instruction memory at address 1 presents 0x15.

At the edge:

```
IR: 0x03 → 0x15  
PC: 1 → 2  
phase: FETCH → EXECUTE
```

Cycle 4 — EXECUTE

The decoder selects R1; the zero-extended immediate is 5.

At the edge:

```
R1: 0 → 5  
phase: EXECUTE → FETCH
```

Instruction 3: ADD R0, R1

Cycle 5 — FETCH

Instruction memory at address 2 presents 0x44.

At the edge:

```
IR: 0x15 → 0x44  
PC: 2 → 3  
phase: FETCH → EXECUTE
```

Cycle 6 — EXECUTE

Between edges:

```
decoder output: alu_op=ADD, reg_write=1, destination=R0  
register outputs: R0=3, R1=5  
ALU inputs: A=3, B=5  
settled ALU output: 8
```

At the edge:

```
R0: 3 → 8  
phase: EXECUTE → FETCH
```

Nothing “travels into R0 one arithmetic step at a time.” Carries propagate through the combinational adder before the edge; then the complete eight-bit result is captured.

Instruction 4: STORE R0, 0

Cycle 7 — FETCH

At the edge:

```
IR: 0x44 → 0x80  
PC: 3 → 4  
phase: FETCH → EXECUTE
```

Cycle 8 — EXECUTE

The decoder selects `R0=8`, address 0, and `mem_write=1`.

At the edge:

```
MEM[0]: 0 → 8  
phase: EXECUTE → FETCH
```

Instruction 5: HALT

Cycle 9 — FETCH

At the edge:

```
IR: 0x80 → 0xC0  
PC: 4 → 5  
phase: FETCH → EXECUTE
```

Cycle 10 — EXECUTE

At the edge:

```
halted: 0 → 1
```

Final architectural result:

```
R0=8 R1=5 R2=0 R3=0 MEM[0]=8
```

This trace covers more than three instructions, but the mastery target is not the quantity. It is the ability to distinguish:

- values that merely settle on combinational wires;
- control decisions that enable a future write;
- and state that actually changes at the next edge.

10. Build and observe the Tiny-8 simulator

The challenge is:

`weekly/challenges/week-005-tiny-cpu.py`

First, predict the final values of `PC`, `IR`, `R0`, `R1`, and `MEM[0]`. Then run the complete trace:

```
python3 weekly/challenges/week-005-tiny-cpu.py
```

Each row exposes:

- state before the edge;
- the active control signals;
- ALU inputs and output when relevant;
- state after the edge;
- a compact list of committed state changes.

Now step manually:

```
python3 weekly/challenges/week-005-tiny-cpu.py --step
```

Before pressing Enter, cover the “after” state and answer:

1. Which combinational outputs should be settling now?
2. Which write enables are active?
3. Which state elements will change at the edge?
4. Which values will remain unchanged even though circuitry is active?

Finally, inspect the source. Locate three boundaries:

```
decode()          combinational control decision
combinational()   candidate values between edges
clock()           committed state update at the edge
```

Python executes these sequentially, whereas real combinational circuits react in parallel and settle over propagation time. The simulator is a state- transition model, not a transistor timing simulation.

11. Common wrong models to repair

“The clock pushes the instruction through the CPU”

The clock coordinates state capture. Voltage changes propagate because circuits respond to their inputs. There is no clock hand carrying bytes.

“The PC is the current instruction”

In this design, after FETCH the PC already points to the next instruction, while the IR preserves the current one. Other architectures define and expose PC behavior differently, so state the convention.

“Decode is where the CPU changes state”

Our decoder is combinational. Its output can change when IR or phase changes, but architectural state changes only when an enabled storage element captures a value.

“The ISA diagram is the hardware”

An ISA specifies behavior. A block diagram describes one organization. A layout contains actual devices and wires. These are related descriptions at different abstraction levels.

“Instruction memory knows that 0x44 is ADD”

Memory returns a pattern. The Tiny-8 decoder interprets fields according to the Tiny-8 encoding. A different reader gives the same pattern another meaning.

“Every instruction always has fetch, decode, execute as three cycles”

Those are logical activities. A CPU may combine them in one clock interval, split them across many intervals, overlap several instructions in a pipeline, or translate instructions internally. The ISA does not normally dictate that schedule.

12. Mastery checkpoint

Close the lesson and answer:

What changes in this CPU between two adjacent clock cycles?

A mastery-level answer must separate three moments:

1. **Just after the first edge:** state elements expose the values captured at that edge.
2. **Between edges:** those outputs drive combinational instruction memory, decoding, selection, and ALU paths; candidate next values and control signals settle, but no enabled state update has yet been committed.
3. **At the next edge:** only enabled state elements capture candidates—for example IR and PC on FETCH, one register or memory location on EXECUTE, or the halted bit for HALT.

It should also say that unchanged registers preserve their values, and that the clock coordinates capture rather than creating the computation.

Explain

Without notes:

- explain why a combinational adder is not a programmable CPU;
- distinguish architectural state from microarchitectural state;
- explain the jobs of PC, instruction memory, IR, register file, ALU, data memory, and control unit;
- distinguish datapath from control;
- distinguish ISA specification, instruction byte, hardware decoder, and microarchitecture;
- explain why “fetch-decode-execute” need not mean three cycles.

Draw

Recreate the minimal CPU block diagram. Your arrows must show:

- PC to instruction-memory address;
- instruction-memory byte to IR;

- IR fields to the control unit and register selectors;
- register outputs to ALU;
- ALU result back to register write data;
- register value and IR address field to data memory;
- control signals reaching every state element they enable.

Beside the diagram, draw two vertical clock edges. In the space between them, write the combinational work for `ADD R0, R1`. At the second edge, mark only `R0` and `phase` as changing.

Observe

Run both normal and `--step` simulator modes. Capture one `FETCH` transition and one `ADD` transition. For each, record:

state before	combinational/control activity	enabled writes	state after
--------------	--------------------------------	----------------	-------------

Explain why the ALU may display a value even on a cycle where no register captures it.

Build

1. Re-encode the sample program by hand and compare it with the simulator.
2. Change the immediates to compute `9 + 6`, storing at address 3.
3. Complete the intentional extension described in the challenge: add a zero flag, or design a branch instruction and state exactly which existing encoding must change. Do not silently overload a bit field.

Why? notebook

1. Why does the PC advance during `FETCH` rather than because “time passed”?
2. Why is an IR useful after the PC changes?
3. Why can the ALU be active when no architectural state changes?
4. Why does a register need both data input and write enable?
5. Why is the decoder hardware rather than meaning attached to memory?
6. Why can two CPUs implement Tiny-8 differently yet run the same byte program?
7. Why must the clock period account for combinational propagation delay?
8. Why is the phase register microarchitectural in this design?
9. Where, physically, does `3 + 5 = 8` happen, and when does 8 become durable CPU state?

Mental model at the end

```
stored instruction pattern selected by PC
    ↓ instruction-memory combinational read
IR captures the pattern at a clock edge
    ↓ decoder and register reads settle
control selects a datapath operation
    ↓ combinational ALU/address work
candidate result becomes stable
    ↓ next enabled clock edge
selected architectural state captures the result
    ↓
PC identifies the next stored instruction
```

The CPU is not a box in which the clock makes computation happen. It is a network of state elements separated by combinational paths. Stored instructions cause the control network to select paths and write enables. Repeated, coordinated state transitions turn fixed circuits into a programmable machine.

References used selectively

Local reading:

- Yale N. Patt and Sanjay J. Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 4, “The von Neumann Model,” for memory, processing unit, input/ output, control unit, instruction cycle, PC, and IR; and Chapter 5, “The LC-3,” for the distinction between an ISA specification, encoded instructions, registers, memory, and instruction behavior. Local PDF: [source-materials/library/books/01-foundations-and-architecture/introduction-to-computing-systems-patt-patel-2e.pdf](#)
- Charles Petzold, *Code: The Hidden Language of Computer Hardware and Software*, Chapters 14, 16, and 17—“Feedback and Flip-Flops,” “An Assemblage of Memory,” and “Automation”—for the progression from remembered state to memory and automatic instruction sequencing; Chapter 18, “From Abaci to Chips,” for historical context. Local PDF: [source-materials/library/books/01-foundations-and-architecture/code-hidden-language-computer-hardware-software-charles-petzold.pdf](#)

Stored-program history:

- University of Manchester Department of Computer Science, “History and heritage,” on the Small-Scale Experimental Machine and its first successful stored-program run: <https://www.cs.manchester.ac.uk/about/history-and-heritage/>
- University of Manchester Computer Science curation, “The First Program,” describing Tom Kilburn's factor program, its instruction count, and the Baby's hardware subtractor: <https://curation.cs.manchester.ac.uk/digital60/www.digital60.org/birth/program/firstprog.html>

The Tiny-8 ISA and simulator are teaching designs created for this lesson. They are intentionally small enough that every instruction bit, control decision, combinational result, and state update can be inspected.